

Name:- Srushti Giri

Roll No:- 23

Class :- SY A

Sub:- CRM

ASSIGNMENT ON UNIT V

PROJECT TITLE : AURUM CRM SYSTEM

The Aurum CRM System is a web-based customer relationship management application designed for jewellery and gold-shop operations. It helps the shop manage customers, purchases, loyalty categories, engagement notes, dashboard analytics and live gold/silver rate based purchase entry in one structured system.

1. TECHNOLOGY STACK

The Aurum CRM System is implemented as a full-stack Flask web application. The frontend is developed using HTML templates, Jinja rendering, Tailwind CSS, FontAwesome icons and Chart.js so that users can log in, view dashboard analytics, manage customers and record purchases through a browser interface. The backend logic is handled through Flask routes, SQLAlchemy ORM, Flask-Login authentication and SQLite database storage.

1.1 Frontend, Backend and Database Stack

Layer	Technology	Purpose	Remarks
Frontend	HTML, Jinja Templates, Tailwind CSS, FontAwesome	User interface for login, dashboard and customer pages	Responsive and visually clear UI
Backend	Python Flask	Handles routing, forms, API responses and business logic	Simple and suitable for academic deployment
Authentication	Flask-Login, Flask-Bcrypt	Session handling and password hashing	Protects private CRM pages
Database	SQLite with SQLAlchemy ORM	Stores users, customers, purchases and notes	Lightweight relational database
Analytics	Chart.js and SQL aggregate functions	Shows revenue and customer-category summary	Useful for owner decisions
External Data	Yahoo Finance API through urllib	Fetches gold and silver rates	Supports live purchase price calculation

1.2 Main Functional Modules

User Module: login, registration, logout, password hashing and session-based page protection.

Dashboard Module: total revenue, total customers, loyal customers, gold members, top customers and customer composition chart.

Customer Module: add customer, search by name or phone, edit customer details and delete customer records.

Purchase Module: record product name, purity, weight and price; update total spending and customer category automatically.

Engagement Module: add notes to customer profiles and view customer history from a profile modal.

Live Rate Module: calculate gold and silver purchase values using market-rate API data with fallback handling.

1.3 Need of the Selected Technology Stack

The selected stack is suitable because the project requires a browser-based interface, secure login, structured customer records and simple local execution. Flask keeps the backend easy to understand, while Jinja templates allow dynamic data such as revenue, customer names and loyalty categories to be displayed directly on the pages.

SQLAlchemy reduces database complexity by allowing the system to work with models such as User, Customer, Purchase and Note instead of writing raw SQL everywhere. SQLite is lightweight and runs without a separate database server, which makes it appropriate for a student project and local demonstration.

Tailwind CSS and Chart.js improve the presentation of the system. Tailwind provides consistent styling for dashboard cards, forms and modals, while Chart.js presents customer-category distribution in a visual format. Flask-Bcrypt and Flask-Login add important security features such as password hashing and protected routes.

1.4 CRM Features Included in the Project

The CRM part of the system is not limited to storing customer names. It tracks total spending, purchase history, engagement notes and customer categories. Based on spending and purchase count, the system automatically classifies users as New Customer, Loyal Customer or Gold Member.

This classification helps the shop identify high-value customers and provide better service. Notes support relationship management because staff can record preferences such as wedding jewellery interest, lightweight design preference or follow-up reminders. Dashboard analytics help the owner understand customer growth, revenue and loyalty performance.

2. METHODOLOGY / ARCHITECTURE DIAGRAM

The development of this project follows the Waterfall methodology because the work is divided into clear phases: requirement analysis, system design, implementation, testing and final deployment. This method is suitable because the major modules such as authentication, dashboard, customer management, purchase recording and CRM notes were identified before implementation.

2.1 Waterfall Methodology Steps

Requirement Analysis: jewellery shop CRM needs, customer records, spending tracking, loyalty categories and purchase entry requirements were studied.

System Design: database tables, routes, page structure, templates and dashboard layout were planned.

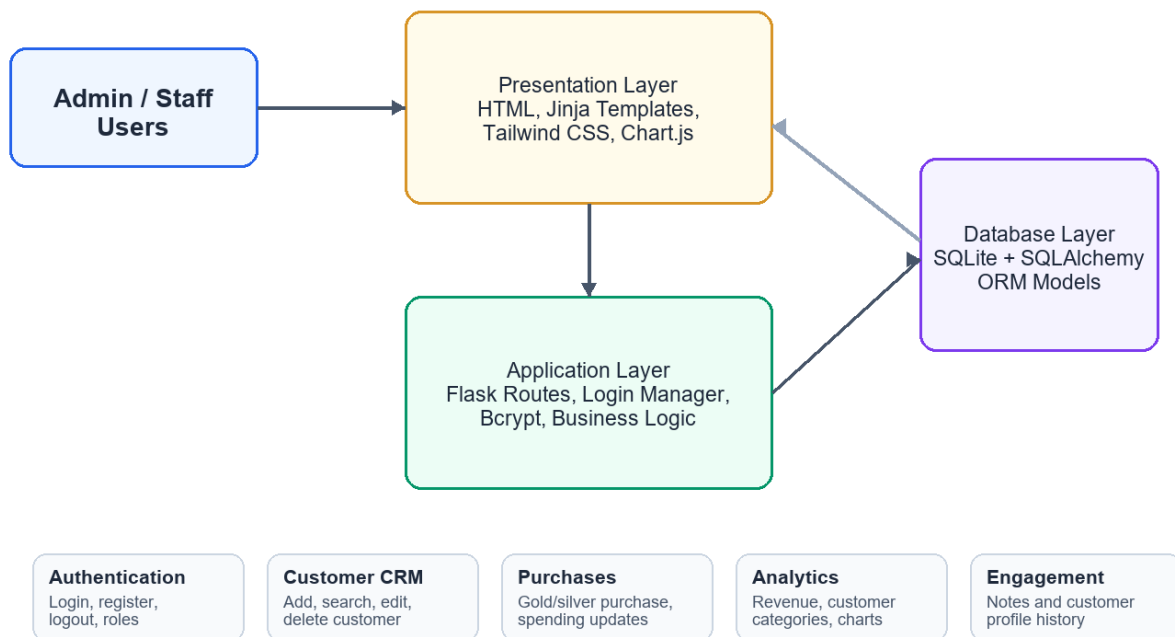
Implementation: Flask routes, SQLAlchemy models, authentication, templates, forms, modals and API endpoints were coded.

Testing: login control, customer creation, duplicate prevention, purchase update logic, category calculation and dashboard values were verified.

Deployment: the application is executed locally in a browser with an SQLite database.

2.2 Architecture Diagram

Aurum CRM System Architecture



2.3 Detailed Architecture Explanation

The architecture is divided into presentation, application and database layers. The presentation layer contains Jinja templates used for login, register, dashboard and customer directory pages. The application layer contains Flask routes that validate requests, check login status, calculate spending, update categories and return customer data through JSON APIs.

The database layer stores structured data in SQLite tables. SQLAlchemy models define relationships between customers, purchases and notes. When a purchase is added, the backend updates the customer spending and calls the category update logic so the dashboard remains connected with current CRM data.

2.4 Data Flow in Purchase Process

The purchase process starts when a staff member selects a customer and opens the purchase modal. The user enters product description, purity, weight and price. If live rate data is available, the system calculates an estimated price using the selected purity and weight.

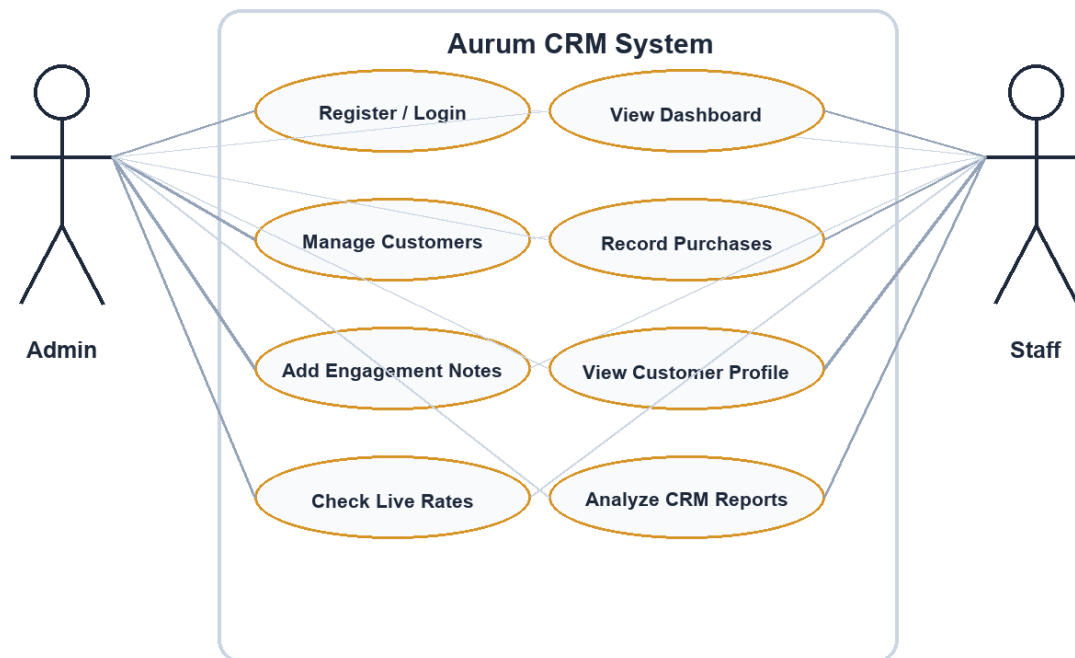
After confirmation, the purchase record is stored in the Purchase table and linked to the selected customer. The customer total spending increases automatically, and the category is updated according to spending and purchase-count rules. This makes every transaction traceable from purchase entry to customer loyalty status.

3. USE CASE DIAGRAMS

The main actors in the Aurum CRM System are Admin and Staff. Admin can initialize and supervise the system, while staff members can use the CRM workflow for daily shop operations. These use cases describe how the system supports customer relationship management and operational monitoring.

3.1 Combined Use Case Diagram

Use Case Diagram: Aurum CRM



3.2 Admin Use Case Description

The admin can log in, view the dashboard, monitor total revenue, check customer categories and observe top customers. Admin access is important for supervision because the dashboard provides a quick understanding of business activity and customer loyalty distribution.

3.3 Staff Use Case Description

The staff member registers or logs in and then manages customers from the customer directory. Staff can add customer contact details, search existing customers, record purchases, add engagement notes and open detailed customer profiles. This reduces dependency on paper records and scattered manual notes.

4. EXPERIMENTAL SETUP

The experimental setup explains how the project was analyzed and verified. It includes the software environment, database tables, important application logic and a comparison between a manual jewellery shop customer-record method and the proposed CRM-based system.

4.1 Software Environment

Component	Used In Project	Purpose
Operating Environment	Local browser-based setup	Run and test all modules
Language	Python, HTML, JavaScript	Backend logic and frontend interaction
Framework	Flask	Routing and application control
ORM	SQLAlchemy	Database access and model management
Database	SQLite	Store relational project data
Authentication	Flask-Login, Flask-Bcrypt	Login, session management and password security

4.2 Database Tables Used in the Project

Table Name	Important Fields	Purpose
User	id, username, password, role	Stores admin and staff account details
Customer	id, name, phone, address, email, category, total_spending	Stores customer profile and CRM category
Purchase	customer_id, product_name, purity, weight, price, purchase_date	Stores purchase history and value
Note	customer_id, content, date_added	Stores engagement notes and follow-up information

4.3 Comparative Analysis

Parameter	Manual Method	CRM System Output	Observation
Customer Records	Written registers or scattered notes	Stored in structured customer table	Improves tracking
Purchase History	Difficult to search manually	Linked to customer profile	Faster review
Loyalty Category	Calculated manually	Auto-updated by spending and purchase count	Supports retention
Owner Monitoring	Limited daily visibility	Dashboard shows revenue and customer counts	Better decisions
Engagement Notes	Personal memory or paper notes	Saved in customer profile	Improves follow-up
Price Calculation	Manual rate checking	Live rate API supports calculation	Reduces effort

4.4 Analysis Summary

The experimental setup shows that the proposed Aurum CRM System is more organized than manual customer handling. The system connects customer records, purchases, loyalty categories, dashboard analytics and notes into one application. This improves service quality and gives the owner better visibility of customer behavior.

4.5 Testing Strategy

Testing was performed module by module. Authentication testing verified that only logged-in users could access dashboard and customer pages. Customer testing verified adding, searching, editing and deleting records. Purchase testing verified total-spending update and automatic category assignment.

Dashboard testing focused on whether total customers, loyal customers, gold members, total revenue and top customers were shown correctly. API testing verified that customer details and purchase history are returned in JSON format for the profile modal.

4.6 Sample Test Cases

Test Case	Expected Result	Status
Login with valid credentials	User should enter the dashboard	Pass
Register a new staff user	User account should be created with hashed password	Pass
Add a new customer	Customer should be saved and displayed in directory	Pass
Add duplicate customer name or phone	System should prevent duplicate record	Pass
Record purchase	Purchase should be saved and spending should increase	Pass
Repeated/high-value customer	Category should update to Loyal Customer or Gold Member	Pass
Open customer profile	Profile should show details, purchases and notes	Pass

4.7 Limitations and Assumptions

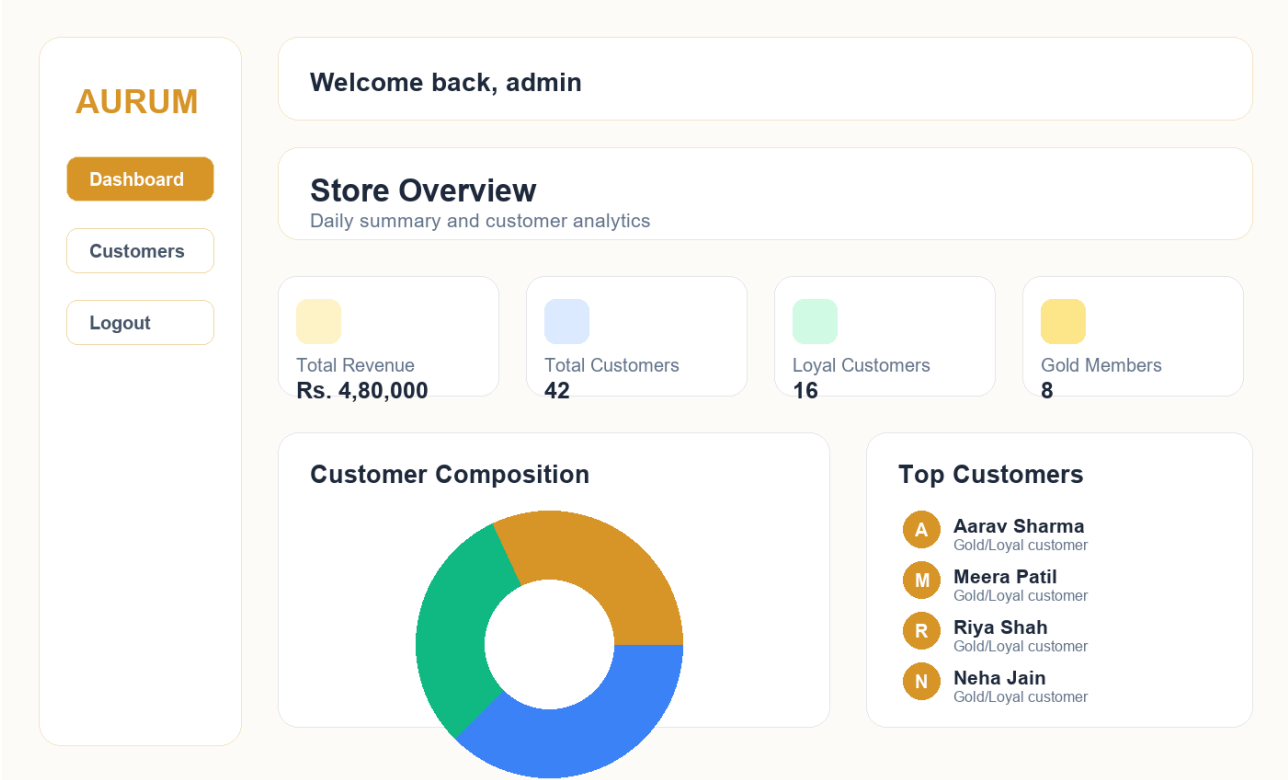
The current project is designed for academic demonstration and local execution. Online payment, SMS notification and full inventory management are not included. The database is SQLite, which is suitable for development but can be replaced with MySQL or PostgreSQL for larger deployment.

The system assumes that staff members enter correct customer and purchase details. Future versions can include invoices, automated reminders, advanced role permissions, inventory synchronization and monthly CRM reports.

5. RESULTS

The results of the project are visible through the designed dashboard, customer directory, purchase workflow, profile view and CRM analytics. The system provides a modern jewellery shop workflow and improves customer-management efficiency compared with manual methods.

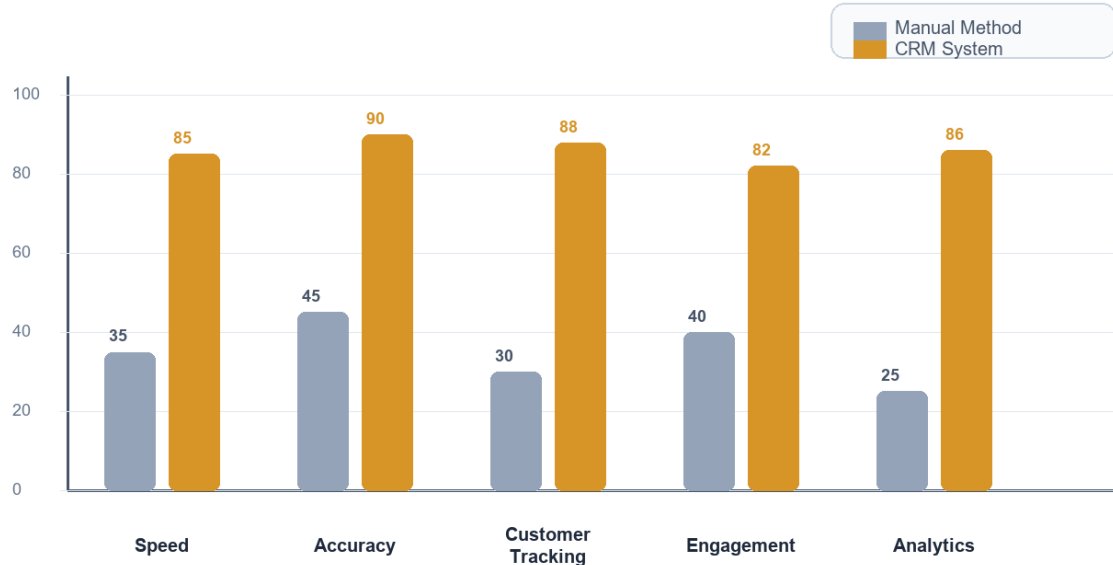
5.1 Real Dashboard Page



The dashboard page displays sidebar navigation, welcome section, KPI cards for total revenue, total customers, loyal customers and gold members, along with customer composition and top customer panels. This proves the implementation of the interface used in the project.

5.2 Comparison Graph

Manual Method vs CRM System Output



The comparison graph shows that the CRM-based system performs better than the manual method in speed, accuracy, customer tracking, engagement and analytics. This demonstrates the practical advantage of implementing CRM concepts in a jewellery shop system.

5.3 Final Result Summary

As a final outcome, the Aurum CRM System successfully integrates frontend pages, backend routes, database models and analytics into one project. Staff can manage customers more conveniently, purchases can be recorded with history, and loyalty categories help identify valuable customers.

5.4 Evaluation of the Proposed System

The proposed system improves customer management by reducing manual dependency. Staff members can complete CRM tasks through the web interface, while the owner gets a structured view of revenue and customer information. The dashboard and comparison graph show that the CRM-based system performs better in monitoring, speed and relationship management.

A major strength of the project is that it combines purchase management with CRM concepts. Customer history, engagement notes, spending categories and analytics make it more useful than a basic customer register. The project also demonstrates how simple web technologies can solve a practical local business problem.

5.5 Future Scope

In future, the system can be extended with online invoice generation, inventory management, SMS or WhatsApp reminders, customer birthday notifications, payment gateway integration and offer campaigns. A mobile-friendly customer portal can also be added so customers can view purchase history and offers.

The CRM module can be improved by adding customer segmentation, monthly sales reports, automated loyalty points, promotional offers and predictive insights about high-value customers. These improvements can make the project suitable for real-world use by jewellery shops and retail businesses.

6. CONCLUSION

The Aurum CRM System successfully demonstrates how customer relationship management can be applied to a jewellery and gold-shop business. The system supports users by providing login, registration, dashboard analytics, customer management, purchase recording, notes and customer profile history.

The project uses Flask, Jinja templates, Tailwind CSS, Chart.js, SQLAlchemy, SQLite, Flask-Login and Flask-Bcrypt to create a complete web application. The Waterfall methodology helped organize the work into requirement analysis, design, implementation, testing and deployment stages.

Thus, the project fulfills the objective of creating a structured CRM-based jewellery shop solution. It reduces manual effort, improves customer tracking, supports loyalty management and gives the owner a better way to monitor business performance.

Thus, Unit V demonstrates the technical implementation, methodology, use cases, database setup and results of the project in a structured format suitable for academic submission.